자바 3일차 수업

목표

➔ 데이터 처리 (연산자) 이해

➔ 형변환이란 변수 또는 상수의 타입을 다른 타입으로 변환하는 것을 말한다.

프로그램에서 **값의 대입이나 연산을 수행할 때는 같은 타입끼리만 가능**하다. 그래서 연산을 수행하기 전에 같은 타입으로 만들어야 하는데, 변수나 상수를 다른 타입으로 변환하는 것을 '**형변환(casting)**' 이라고 한다.

➔ 형변환은 물을 큰 그릇과 작은 그릇 사이에서 옮기는 것을 생각하면 이해가 쉽다. 각 타입을 크기가 다른 그릇이라고 생각해보자.

➔ 형변환은 한 타입(그릇)의 데이터를 다른 데이터 타입(그릇)으로 옮겨 담는 것이다. 형변환에는 개발자가 지정하지 않아도 자동적으로 이루어지는 **자동 형변환**과 개발자가 명시해야만 이루어지는 **강제 형변환**이 있다.

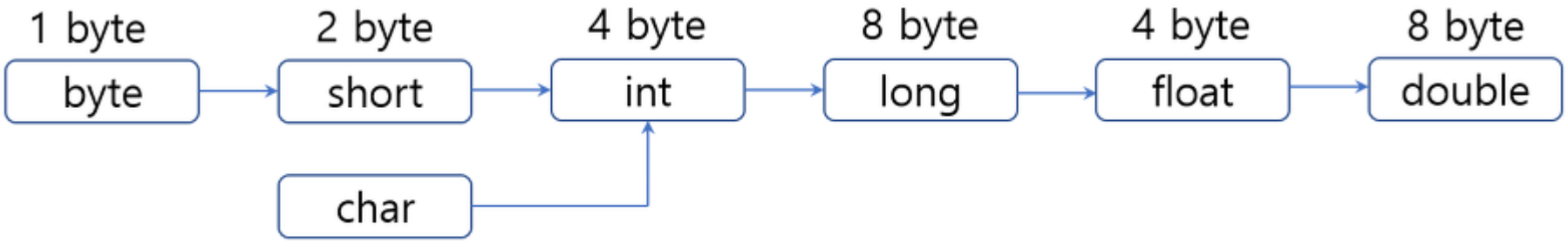
➔ 형변환

형식) **(타입)피연산자**

기본형에서 **boolean**을 제외한 나머지 타입들은 서로 형변환이 가능하다.
기본형과 참조형간의 형변환은 불가능하다.
자동 형변환과 강제 형변환 두가지가 있다.

1) 자동 형변환

➔ 서로 다른 타입간의 대입이나 연산을 할 때 형변환으로 타입을 일치시켜서 하는것이 원칙이다.
➔ 작은 데이터 타입에서 큰 데이터 타입으로 형변환하여도 데이터의 손실 없이 그대로 보존될 수 있기 때문에 자동 형변환이 일어난다.



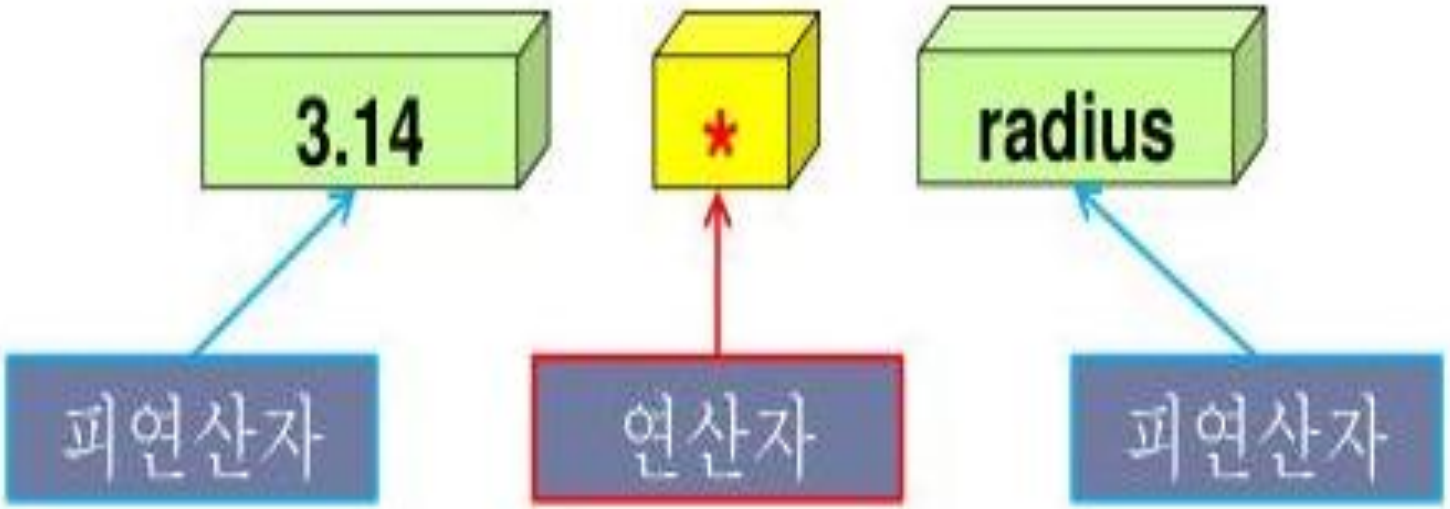
자동 형변환이 가능한 방향

2) **강제 형변환 (명시적 형변환)**

➔ 큰 데이터 타입에서 작은 데이터 타입으로 옮길 때 데이터의 손실이 발생할 수도 아닐 수도 있다.
데이터의 손실이 일어난다면 정확한 연산을 수행할 수 없기 때문에 예상하지 못한 결과를 얻을 수 있다.
그러므로 강제 형변환시 주의해서 형변환해야 한다.

변 환	수 식	결 과
int → char	(char)65	'A'
char → int	(int)'A'	65
float → int	(int)1.6f	1
int → float	(float)10	10.0f

- 수식(expression)
  - 상수, 변수, 연산자의 조합.
  - 연산자와 피연산자로 나누어진다.
    - 피연산자- 연산의 대상
    - 연산자- 어떤 연산을 나타내는 기호



기능에 따른 연산자의 분류

연산자의 분류	연산자	의미
대입	=	오른쪽을 왼쪽에 **대입**
산술	+ - * / %	사칙연산과 나머지 연산
부호	+ -	양수와 음수 표시
증감	++ --	증가, 감소 연산
관계	> < == != >= <=	오른쪽과 왼쪽을 비교
논리	&& || !	논리적인 AND, OR
조건	?	**조건**에 따라 선택
콤마	,	피연산자들을 **순차적**으로 실행
비트 단위 연산자	& | ^ ~ << >>	비트별 AND, OR, XOR, 이동, 반전
sizeof 연산자	sizeof	자료형이나 변수의 크기를 바이트 단위로 반환
형변환	(type)	변수나 상수의 자료의 형을 변환
포인터 연산자	* & [ ]	주소계산, 포인터가 가리키는 곳의 내용 추출
구조체 연산자	. ->	구조체의 멤버 참조

피연산자수에 따른 연산자 분류

- 단항 연산자: 피연산자의 수가 1개

```
++x;
--y;
```

- 이항 연산자: 피연산자의 수가 2개

```
x + y
x - y
```

- 삼항 연산자: 연산자의 수가 3개

```
x ? y : z
```

산술 연산자

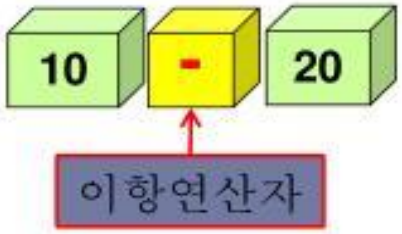
- 산술 연산: 컴퓨터의 가장 기본적인 연산
- 덧셈, 뺄셈, 곱셈, 나눗셈 등의 사칙 연산을 수행하는 연산자

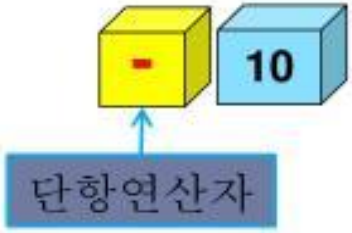
연산자	기호	의미
덧셈	x+y	x와 y를 더한다
뺄셈	x-y	x에서 y를 뺀다.
곱셈	x*y	x와 y를 곱한다.
나눗셈	x/y	x를 y로 나눈다.
나머지	x%y	x를 y로 나눌 때의 나머지값

부호 연산자

- 변수나 상수의 부호를 변경

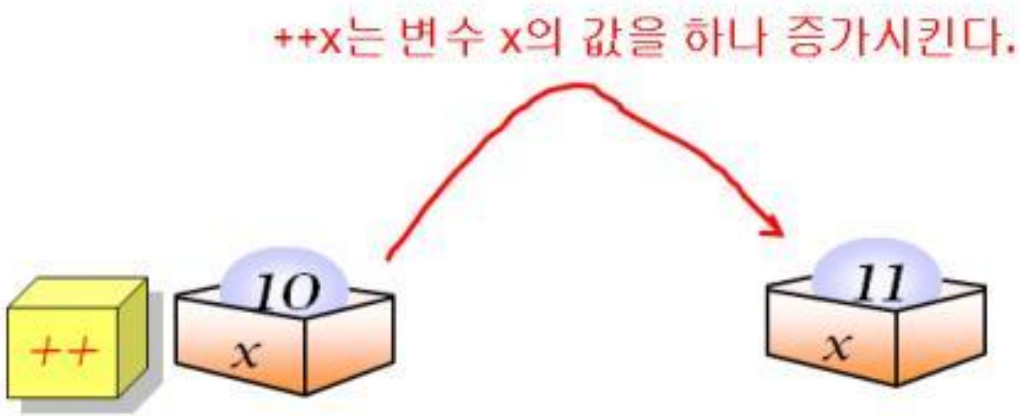
```
x = -10;
y = -x;   // 변수 y의 값은 10이 된다.
```





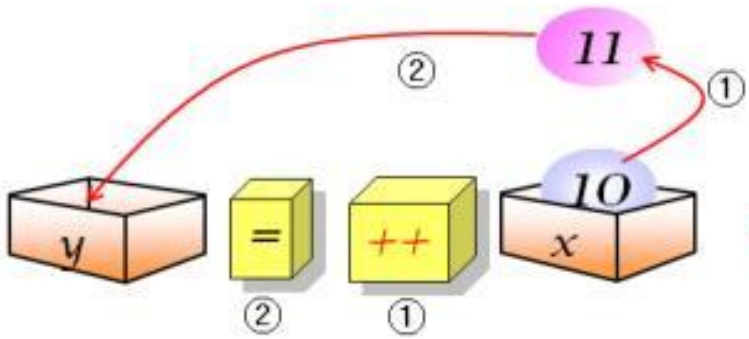
증감 연산자

- 증감 연산자: ++, --
- 변수의 값을 하나 증가시키거나 감소시키는 연산자



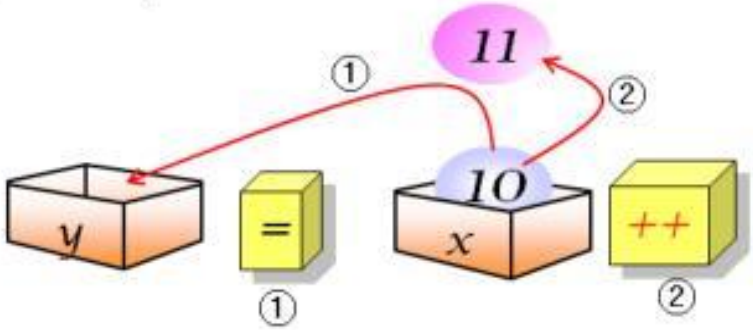
++x와 x++의 차이

- ++x와 x++는 어떤 차이가 있을까?
- y = ++x



증가된 x의 값이 y에 대입된다.

- y= x++;



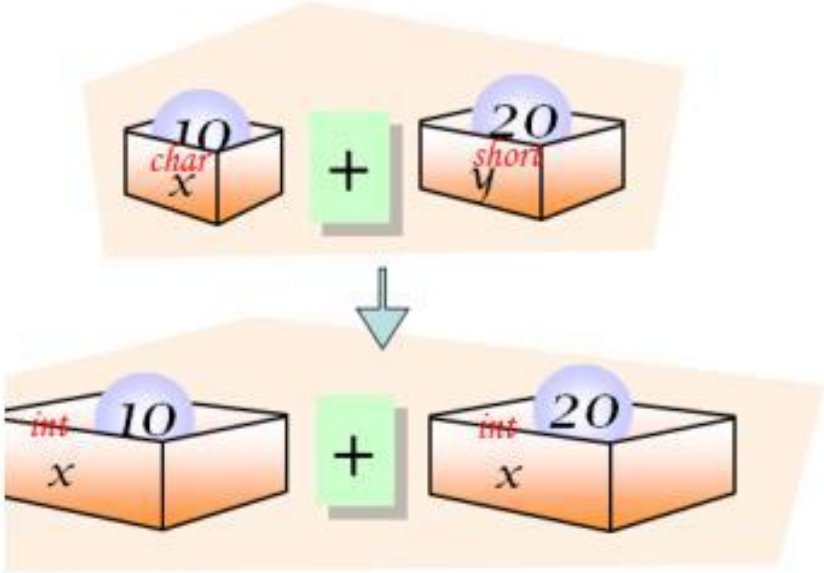
증가되지 않은 x의 값이 y에 대입된다.

증감 연산자 정리

증감 연산자	의미
++x	수식의 값은 증가된 x값이다.
x++	수식의 값은 증가되지 않은 원래의 x값이다.
--x	수식의 값은 감소된 x값이다.
x--	수식의 값은 감소되지 않은 원래의 x값이다.

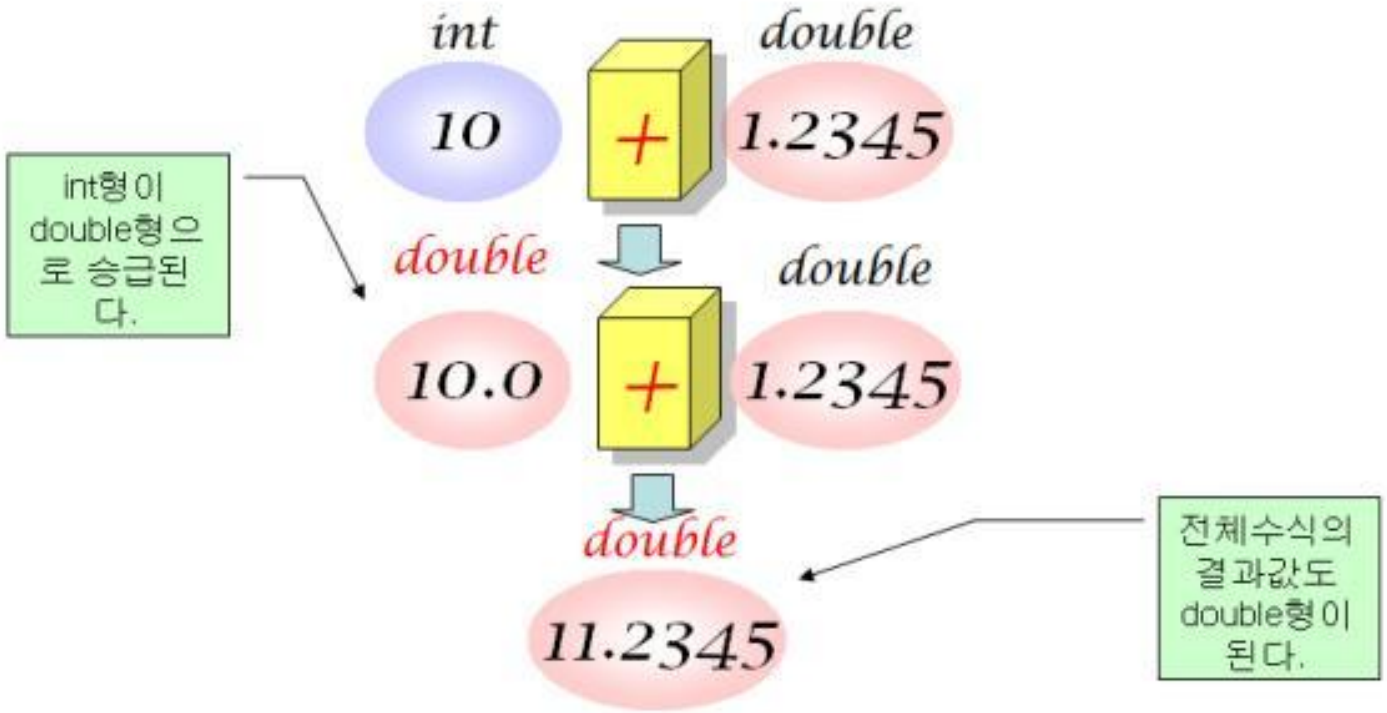
정수 연산시의 자동적인 형변환

- 정수 연산시 char형이나 short형의 경우, 자동적으로 int형으로 변환하여 계산한다.



수식에서의 자동적인 형변환

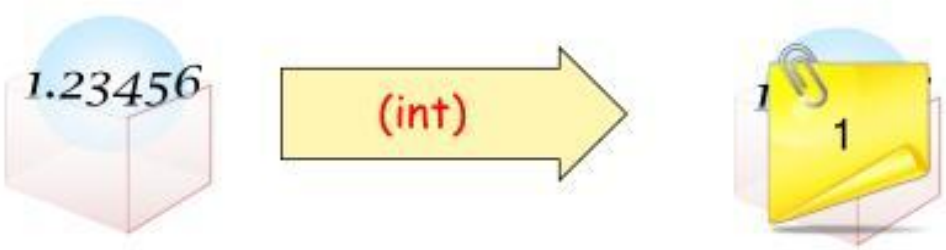
- 서로 다른 자료형이 혼합하여 사용되는 경우, 더 큰 자료형으로 통일된다.



명시적인 형변환

- 형변환(type cast): 사용자가 데이터의 타입을 변경하는 것

- (int) 1.23456 // double형으로 변환
- (double) x // double형으로 변환
- (long) (x+y) // long형으로 변환



관계 연산자

- 두개의 피연산자를 비교하는 연산자
- 결과값은 참(1) 아니면 거짓(0)

x 와 y의 값이 같은지 비교한다.

```
x == y
```

관계 연산자

연산자	의미
x == y	x와 y가 같은가?
x != y	x와 y가 다른가?
x > y	x가 y보다 큰가?
x < y	x가 y보다 작은가?
x >= y	x가 y보다 크거나 같은가?
x <= y	x가 y보다 작거나 같은가?

논리 연산자

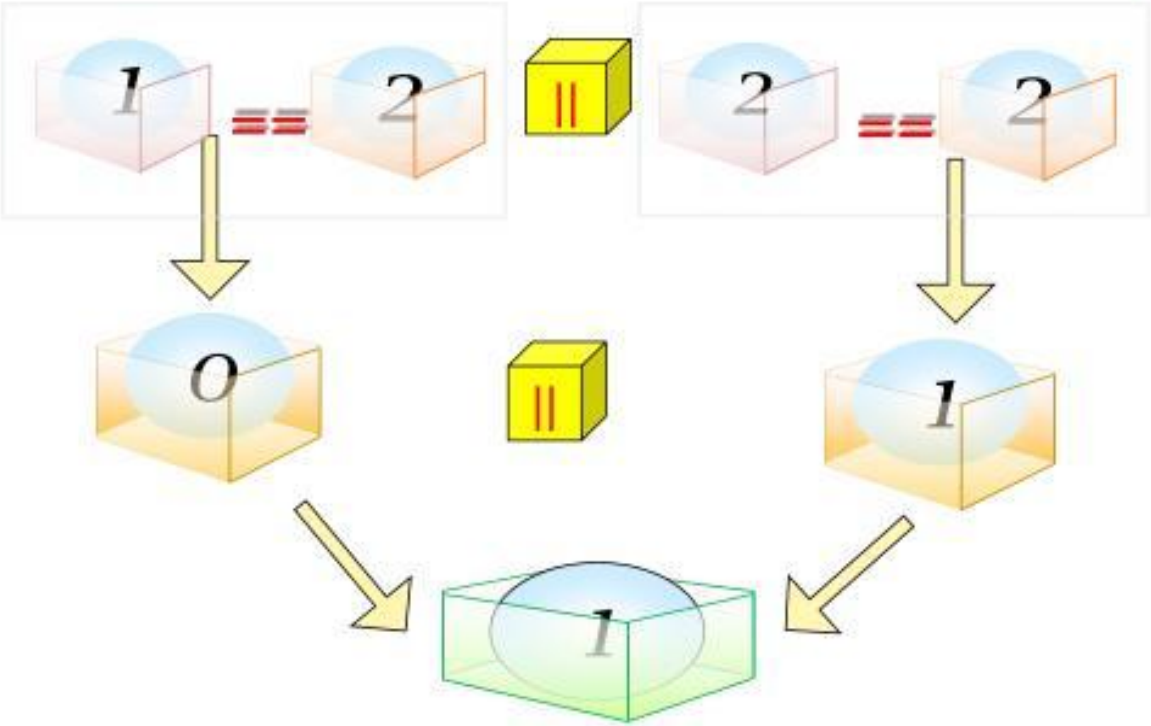
- 여러 개의 조건을 조합하여 참과 거짓을 따지는 연산자
- 결과값은 참(1) 아니면 거짓(0)

연산자	의미
x && y	AND 연산, x와 y가 모두 참이면 참, 그렇지 않으면 거짓
x || y	OR 연산, x나 y중에서 하나만 참이면 참, 모두 거짓이면 거짓
!x	NOT 연산, x가 참이면 거짓, x가 거짓이면 참

X	Y	X && y	x || y
참	참	참	참
참	거짓	거짓	참
거짓	참	거짓	참
거짓	거짓	거짓	거짓

논리 연산자의 계산 과정

- 논리 연산의 결과값은 항상 1 또는 0이다.
- (예) (1 == 2) || (2 == 2)



단축 계산

- && 연산자의 경우, 첫번째 피연산자가 거짓이면 다른 피연산자들을 계산하지 않는다.

```
( 2 > 3 ) && ( ++x < 5 )
```

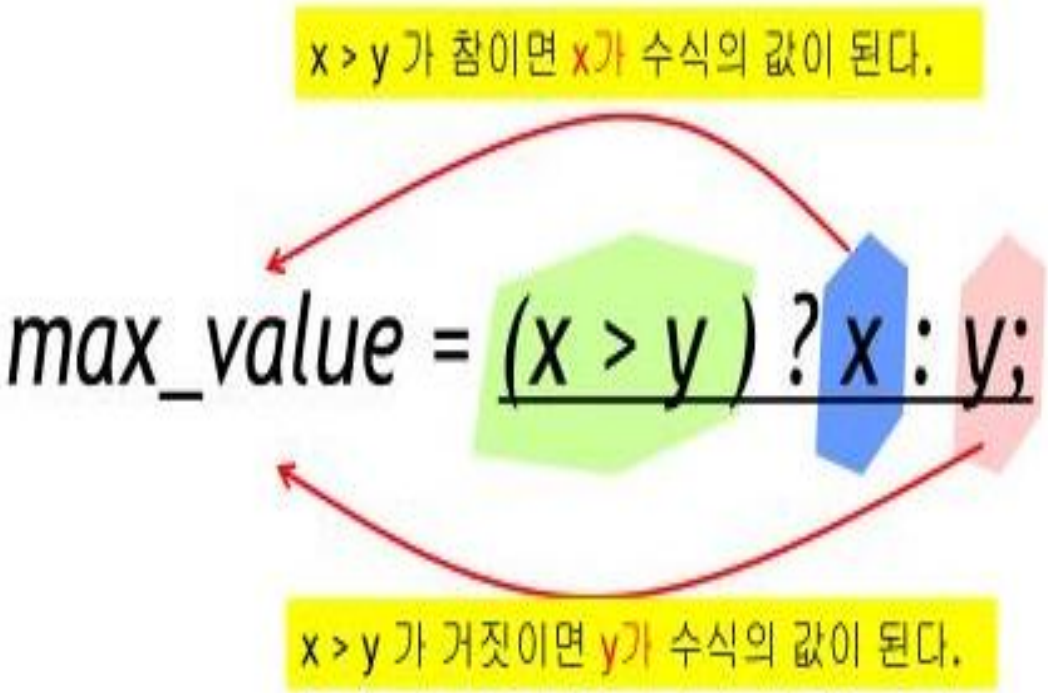
- || 연산자의 경우, 첫번째 피연산자가 참이면 다른 피연산자들을 계산하지 않는다.

```
( 3 > 2 ) || ( --x < 5 )
```

첫번째 연산자가 거짓이면 다른 연산자는 계산할 필요가 없겠군!!

++나 --는 실행이 안될 수도 있으니 주의하세요.

조건 연산자



콤마 연산자

- 콤마로 연결된 수식은 순차적으로 계산된다.

먼저 계산된다.

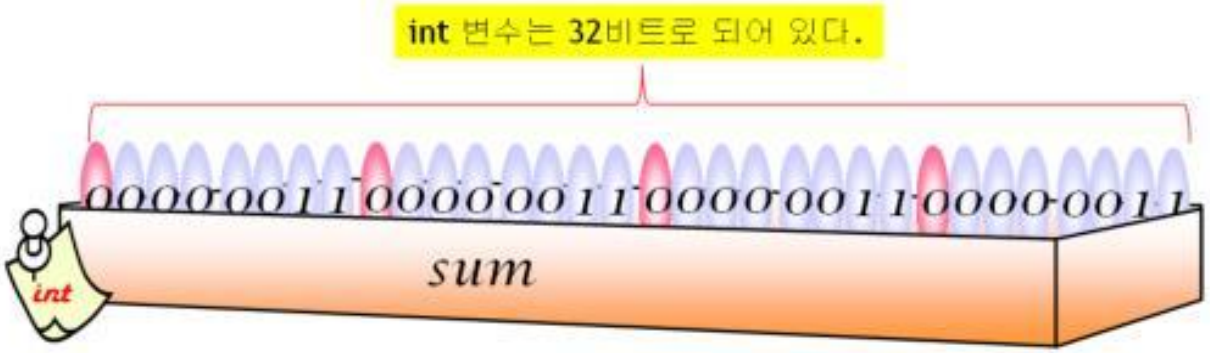
나중에 계산된다.

```
x++, y++ ;
```

비트 연산자

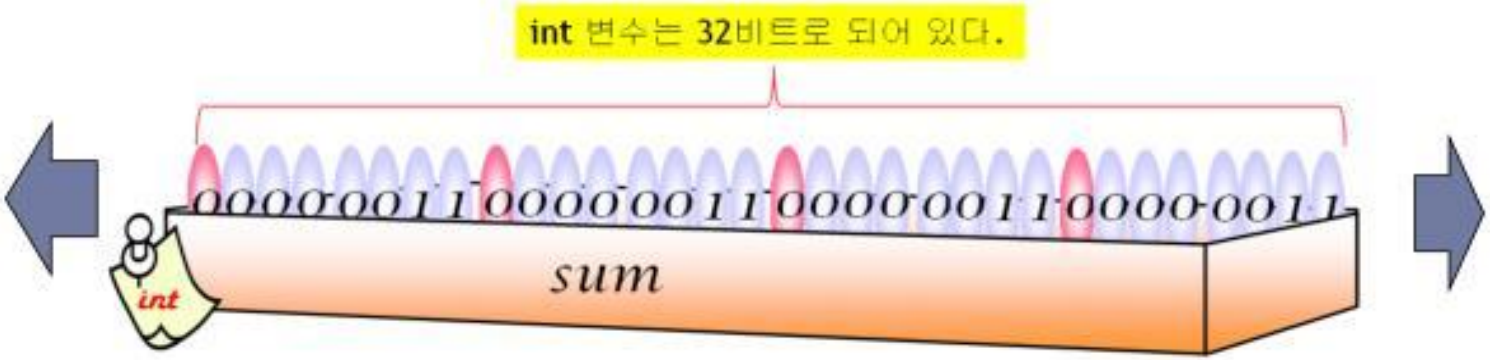
연산자	연산자의 의미	설명
&	비트 AND	두개의 피연산자의 해당 비트가 모두 1이면 1, 아니면 0
|	비트 OR	두개의 피연산자의 해당 비트중 하나만 1이면 1
^	비트 XOR	두개의 피연산자의 해당 비트의 값이 같으면 0, 아니면 1
<<	왼쪽으로 이동	지정된 개수만큼 모든 비트를 왼쪽으로 이동한다.
>>	오른쪽으로 이동	지정된 개수만큼 모든 비트를 오른쪽으로 이동한다.
~	비트 NOT	0은 1로 만들고 1은 0로 만든다.

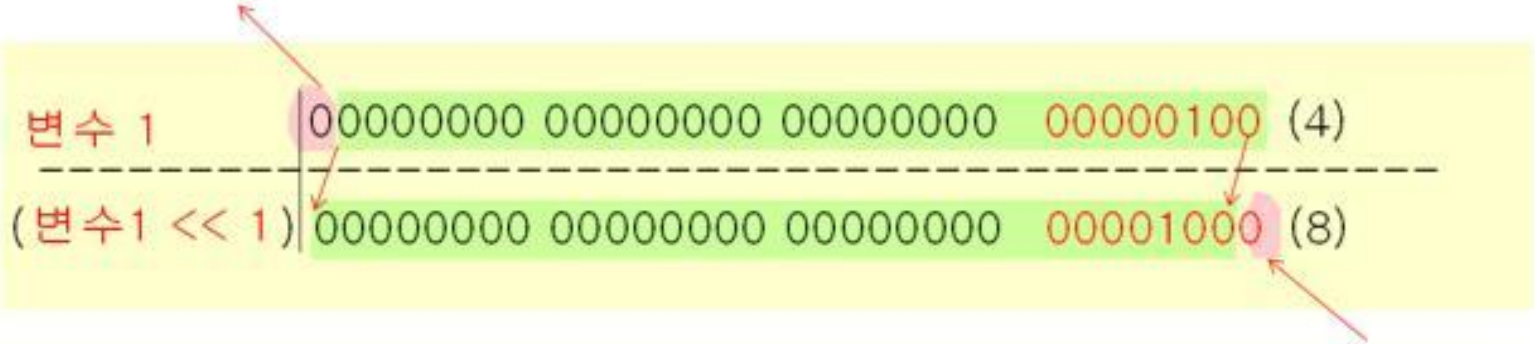
모든 데이터는 비트로 이루어진다.



비트 이동 연산자

연산자	기호	설명
왼쪽 비트 이동	<<	x << y x의 비트들을 y 칸만큼 왼쪽으로 이동
오른쪽 비트 이동	>>	x >> y x의 비트들을 y 칸만큼 오른쪽으로 이동



<< 연산자

- 비트를 왼쪽으로 이동
- 값은 2배가 된다.